

Medicaid Claims Checker

Production Readiness Review

Generated: April 24, 2026

This document provides a comprehensive code review of the Medicaid Claims Checker project, evaluating architecture, code quality, and production readiness. It includes security assessment, observability analysis, and a prioritized roadmap for production hardening.

Executive Summary

The Medicaid Claims Checker is a well-architected system combining a pure functional Haskell engine with a Phoenix LiveView web application. The codebase demonstrates excellent separation of concerns, type safety, and concurrent design patterns.

However, several critical production gaps exist that must be addressed before handling real healthcare data. This review identifies 6 major areas requiring attention and provides a prioritized roadmap for hardening.

Strengths

Well-Architected Separation of Concerns

Haskell engine isolates deterministic business logic from web state. Clear module organization (Claims.* namespace) with single-responsibility modules. Functional purity guarantees reproducibility.

Type Safety & Error Handling

Strong typing prevents runtime errors. Contract validation enforces request schema. Graceful ML fallback handles service failures.

Concurrency & Thread Safety

STM-based rule cache is thread-safe without locks. JSON-first evaluator avoids parsing overhead. LiveView handles concurrent connections cleanly.

Good Test Coverage

Property tests verify parser correctness. Hspec unit tests cover evaluation flows. Test date fixed for deterministic results.

Real-Time UI with PubSub

Phoenix PubSub broadcasts progress during batch operations. Interactive feedback without full page reloads.

Critical Production Gaps

Security

- No authentication/authorization: Any client can submit unlimited requests
- No rate limiting: Risk of DoS attacks on the Haskell engine
- No input validation: Untrusted rule text could exhaust memory or hang parser
- No TLS/HTTPS: Communication is unencrypted
- SFTP credentials stored in plaintext in database

Observability & Logging

- Haskell engine has no logging: Cannot diagnose production failures
- Scattered Logger calls in Phoenix: No standardized correlation IDs
- No metrics/monitoring: Cannot track SLA compliance or error rates

Data Durability & Transactions

- Batch ingestion lacks idempotency: Retries create duplicates
- Evaluation results not persisted: Lost on process crashes
- X12Translator webhook has no retry logic or timeout handling

Configuration & Secrets

- Haskell port hardcoded to 8080: No environment configuration
- Phoenix prod config incomplete: Missing SSL redirect and endpoint URL
- No secrets rotation mechanism: SECRET_KEY_BASE is static

Testing & Deployment

- No integration tests: HTTP endpoints not tested end-to-end
- No load testing: Unknown production throughput capacity
- No deployment runbook: Unclear how to safely deploy updates

Error Handling

- Inconsistent error response formats: Some return JSON, some plain text
- Missing fields silently fail without logging: Typos in rules undetected
- Transaction failures have no retry logic

Production-Readiness Checklist

This table prioritizes the work needed to reach production readiness. Critical items must be completed before handling real data; high-priority items should be completed in Phase 1.

Item	Status	Priority	Description
API Authentication	NO	CRITIC	Add OAuth2/API key validation
Rate Limiting	NO	CRITIC	100 req/sec per API key
Input Validation	PARTIAL	HIGH	Rule size limits, parser timeout
Structured Logging	PARTIAL	HIGH	Request ID correlation
Metrics/Monitoring	NO	HIGH	Prometheus integration
Data Durability	PARTIAL	HIGH	Persist results, retry logic
Secret Management	PARTIAL	MEDIUM	Encrypt SFTP credentials
HTTPS/TLS	PARTIAL	MEDIUM	Reverse proxy termination
Integration Tests	NO	MEDIUM	HTTP endpoint tests
Load Testing	NO	MEDIUM	Throughput baseline
Deployment Runbook	NO	MEDIUM	Blue-green, rollback docs
Graceful Shutdown	PARTIAL	MEDIUM	Drain in-flight requests

Recommended Implementation Roadmap

Phase 1: Pre-Production (2 weeks)

- Add API authentication + rate limiting
- Implement structured logging with request IDs
- Set rule size/timeout limits; add error telemetry

Phase 2: Hardening (2 weeks)

- Add integration tests; run load test baseline
- Implement metrics collection (Prometheus)
- Encrypt secrets; finalize deployment runbook

Phase 3: Monitoring (1 week)

- Deploy to staging; run 48-hour soak test
- Set up alerts (CPU, memory, error rate, latency)
- Document on-call runbook (troubleshooting, rollback)

Code Quality Observations

What's Good

Parser.hs

Well-documented with clear grammar examples and usage patterns

RuleEngine.hs

Excellent module docs; clear usage examples for integration

EvalEnv Pattern

Carrying evaluation context through recursion is elegant and safe

What Needs Attention

Main.hs

Ensure all error responses include request_id for debugging; add request logging

claims.ex

Manual batch registration via ETS is fragile; consider using database

router.ex

Add production auth guard for LiveDashboard; document security assumptions

Conclusion

The Medicaid Claims Checker codebase is well-engineered with excellent architecture and code quality. The Haskell engine is production-grade for rule evaluation, and the Phoenix web layer is well-structured.

However, security, observability, and deployment readiness gaps must be addressed before handling real healthcare data. The main risks are:

- Unauthorized access to sensitive batch results
- Blind spot on production failures (no logging/metrics)
- Data loss on crashes (no result persistence)
- Uncontrolled resource usage (no rate limiting or timeouts)

With the Phase 1-3 hardening work outlined in this review, this system will be ready for production. The estimated effort is 3-4 weeks for a small team.

Report generated by AI Code Review Agent